

Documentación — Fase 3

Modelado, Planificador e Integración

Proyecto: LogísTEC (LGTC)

Curso: CE1103 — Algoritmos y Estructuras de Datos I

Fecha: Junio 2026

Instituto Tecnológico de Costa Rica
Escuela de Computación

Índice general

1. Resumen del trabajo realizado	3
2. Estructura de paquetes	4
3. Clases de modelo de dominio (model/)	6
3.1. Package.java	6
3.2. Truck.java	6
4. Carga JSON (io/GraphLoader.java)	8
4.1. Dependencia externa	8
4.2. Uso	8
4.3. Formato JSON esperado	8
4.4. Clase resultado GraphData	9
4.5. Validaciones incluidas	9
5. Planificador (planner/RoutePlanner.java)	10
5.1. Asignación best-fit	10
5.2. Nearest Neighbor (NN)	10
5.3. MST-based (2-aproximación para TSP métrico)	11
5.4. Métodos utilitarios	11
6. Aplicación principal (LogisTEC.java)	12
6.1. Flujo de ejecución (ejecutar(jsonPath))	12
6.2. Punto de entrada	13
7. Interfaz gráfica (ui/LogisTECFrame.java)	14
8. Stubs para algoritmos pendientes (algorithms/)	15
9. Modificaciones a archivos existentes	16
9.1. Renombrados (extensión .java)	16

9.2. Package declarations	16
9.3. Import agregado a <code>LinkedList.java</code>	17
9.4. <code>Prim.java</code> — Nuevo constructor	17
10.Caso de prueba (<code>test_cases/caso_base.json</code>)	18
10.1. Especificaciones	18
10.2. Resultado esperado de la asignación best-fit	18
11.Archivos de compilación y ejecución	19
11.1. <code>compile.bat</code>	19
11.2. <code>run.bat</code>	19
11.3. Estructura de directorios (raíz del proyecto)	19
12.Dependencias externas	20
13.Estado de la compilación	21
13.1. Resultados de la ejecución de prueba	21
14.Responsabilidades del equipo	22
14.1. Pendiente (compañeros)	22
14.2. Ya implementado (en la Fase 3)	22

Capítulo 1

Resumen del trabajo realizado

Se reestructuró el proyecto en paquetes Java según lo requerido en el enunciado, se añadieron las clases de modelo de dominio, carga JSON, planificador de rutas (con dos heurísticas), aplicación principal que integra todo el flujo, interfaz gráfica Swing, y un caso de prueba. Los algoritmos que implementarán los compañeros (DFS, Dijkstra, FloydWarshall, Warshall, Kruskal) quedaron como stubs compilables.

Capítulo 2

Estructura de paquetes

La estructura de paquetes del proyecto se organiza de la siguiente manera:

```
src/cr/ac/tec/ce1103/logistec/
+-- LogisTEC.java           <- Aplicacion principal
+-- graph/                  <- TDAs y estructuras base
|   +-- ArrayList.java
|   +-- BFS.java
|   +-- Edge.java
|   +-- HashMap.java
|   +-- HashSet.java
|   +-- LinkedList.java
|   +-- Map.java            <- Interfaz del mapa
|   +-- MinHeap.java
|   +-- Prim.java
|   +-- Set.java            <- Interfaz del conjunto
|   +-- UnionFind.java
+-- algorithms/             <- Stubs para companeros
|   +-- DFS.java
|   +-- Dijkstra.java
|   +-- FloydWarshall.java
|   +-- Kruskal.java
|   +-- Warshall.java
+-- model/                  <- Modelo de dominio
|   +-- Package.java
|   +-- Truck.java
+-- io/                     <- Entrada/Salida
|   +-- GraphLoader.java
+-- planner/                <- Logica de planificacion
```

```
|   +-- RoutePlanner.java
+-- ui/                               <- Interfaz grafica
    +-- LogisTECFrame.java
```

Capítulo 3

Clases de modelo de dominio (model/)

3.1. Package.java

Representa un paquete a entregar.

Cuadro 3.1: Atributos de la clase Package.

Atributo	Tipo	Descripción
id	String	Identificador único (ej. "P01")
destino	String	ID del vértice de destino
peso	int	Peso en kilogramos
prioridad	int	1 (alta), 2 (media), 3 (baja)
rechazado	boolean	Marca si es no entregable

Métodos públicos:

- `getId()`, `getDestino()`, `getPeso()`, `getPrioridad()`
- `isRechazado()`, `marcarRechazado()`

3.2. Truck.java

Representa un camión de la flota.

Cuadro 3.2: Atributos de la clase Truck.

Atributo	Tipo	Descripción
id	String	Identificador (ej. "C01")
capacidad	int	Capacidad máxima en kg
cargaActual	int	Carga acumulada
paradas	ArrayList<String>	IDs de vértices a visitar en orden

Métodos públicos:

- `getId()`, `getCapacidad()`, `getCargaActual()`
- `capacidadLibre()` — capacidad restante
- `puedeLlevar(peso)` — verifica si cabe un paquete
- `agregarCarga(peso)` — incrementa carga acumulada
- `getParadas()`, `setParadas(lista)`
- `porcentajeOcupacion()` — devuelve 0–100

Capítulo 4

Carga JSON (io/GraphLoader.java)

4.1. Dependencia externa

Requiere Gson 2.11+ (incluido en lib/gson.jar).

4.2. Uso

```
1 GraphLoader.GraphData data = GraphLoader.load("test_cases/caso_base.json");
```

Listing 4.1: Ejemplo de carga de datos JSON.

4.3. Formato JSON esperado

```
1 {
2   "ciudad": {
3     "vertices": [
4       {"id": "AA", "tipo": "DEPOT|INTERSECCION", "x": 100, "y": 100},
5       ...
6     ],
7     "aristas": [
8       {"u": "AA", "v": "AB", "distancia": 180},
9       ...
10    ]
11  },
12  "paquetes": [
13    {"id": "P01", "destino": "G", "peso": 5, "prioridad": 1},
14    ...
15  ],
16  "camiones": [
```

```

17     {"id": "C01", "capacidad": 50},
18     ...
19 ]
20 }

```

Listing 4.2: Estructura del archivo JSON de entrada.

4.4. Clase resultado GraphData

Cuadro 4.1: Campos de la clase GraphData.

Campo	Tipo	Descripción
graph	HashMap>Integer, ArrayList>Edge<<	Lista de adyacencia (índices enteros)
vertexIndex	HashMap>String, Integer<	ID de vértice → índice
vertexId	HashMap>Integer, String<	Índice → ID de vértice
depotIndex	int	Índice del depósito
V	int	Número de vértices
vertexX, vertexY	int[]	Coordenadas para dibujo
paquetes	ArrayList>Package<	Paquetes del caso
camiones	ArrayList>Truck<	Flota de camiones

4.5. Validaciones incluidas

- Error si no se encuentra la sección ciudad
- Error si hay múltiples vértices DEPOT
- Error si no hay ningún vértice DEPOT
- Error si una arista referencia a un vértice inexistente
- Error si un paquete referencia a un destino inexistente

Capítulo 5

Planificador

(planner/RoutePlanner.java)

5.1. Asignación best-fit

```
1 Entrada:  lista de paquetes, lista de camiones
2 Proceso:
3   1. Ordenar paquetes por prioridad ascendente, luego peso descendente
4   2. Por cada paquete:
5       a. Buscar el camion con mayor capacidad libre que pueda alojarlo
6       b. Si existe -> asignar (incrementar cargaActual)
7       c. Si no existe -> marcar como rechazado
8 Salida:  camiones con carga actualizada, paquetes marcados como
          rechazados
```

Listing 5.1: Algoritmo de asignación best-fit.

Complejidad: $O(P \times C)$ donde $P = \#paquetes$, $C = \#camiones$.

5.2. Nearest Neighbor (NN)

```
1 Entrada:  distMatrix[V][V], stops[], depotIndex
2 Proceso:
3   1. Partir del deposito
4   2. Mientras queden paradas sin visitar:
5       a. Encontrar la parada no visitada mas cercana a la posicion actual
6       b. Ir a esa parada
7   3. Regresar al deposito
8 Salida:  ruta como ArrayList<Integer> (deposito -> parada1 -> ... ->
          deposito)
```

Listing 5.2: Algoritmo Nearest Neighbor para ruteo.

Complejidad: $O(n^2)$ donde n = número de paradas.

5.3. MST-based (2-aproximación para TSP métrico)

```
1 Entrada:  distMatrix[V][V], stops[], depotIndex
2 Proceso:
3   1. Construir subgrafo con vertices = {deposito} U {paradas}
4   2. Construir MST usando Prim  $O(m^2)$  sobre matriz densa
5   3. Llamar a DFS.preorder() para obtener el recorrido en preorden del
      MST
6   4. El orden de preorden (sin repetir deposito) es la ruta de visita
7   5. Regresar al deposito
8 Salida:  ruta como ArrayList<Integer>, o null si DFS stub no esta
      implementado
```

Listing 5.3: Algoritmo MST-based para aproximación TSP.

Garantía: a lo sumo $2\times$ la distancia óptima (para distancias métricas que cumplen la desigualdad triangular).

Complejidad: $O(m^2)$ donde $m = \#paradas + 1$.

5.4. Métodos utilitarios

- `calcularDistanciaRuta(ruta, distMatrix)` — suma las distancias entre paradas consecutivas
- `obtenerIndicesParadas(truck, vertexIndex)` — convierte IDs string a índices enteros

Capítulo 6

Aplicación principal (LogisTEC.java)

6.1. Flujo de ejecución (ejecutar(jsonPath))

```
1 +-----+
2 | 1. Cargar JSON (GraphLoader.load) |
3 +-----+
4 | 2. Validar alcanzabilidad (Warshall*) |
5 +-----+
6 | 3. Matriz de distancias (FloydWarshall*) |
7 +-----+
8 | 4. MST con Prim |
9 +-----+
10 | 5. MST con Kruskal* |
11 +-----+
12 | 6. Asignar paquetes (best-fit) |
13 +-----+
14 | 7. Planificar rutas por camion |
15 |   a. Nearest Neighbor |
16 |   b. MST-based* (con DFS stub) |
17 |   c. Elegir la mejor |
18 +-----+
19 | 8. Generar reporte final |
20 +-----+
21 | 9. Mostrar interfaz grafica (Swing) |
22 +-----+
```

Listing 6.1: Flujo de ejecución de la aplicación principal.

Nota: Los pasos marcados con * utilizan stubs y muestran “Pendiente de implementación por el equipo” si aún no han sido reemplazados.

6.2. Punto de entrada

```
1 java -cp "out;lib/gson.jar" cr.ac.tec.ce1103.logistec.LogisTEC  
    test_cases/caso_base.json
```

Listing 6.2: Comando para ejecutar la aplicación.

Capítulo 7

Interfaz gráfica (`ui/LogisTECFrame.java`)

Implementada con **Swing** puro (sin JavaFX).

Componentes

- **GraphPanel** — JPanel que dibuja el grafo completo:
 - Aristas como líneas grises con etiquetas de peso
 - Vértices como círculos (rojo = depósito, amarillo = destino con paquete, blanco = intersección)
 - Leyenda en la esquina superior izquierda
- **JTextArea inferior** — texto informativo con resumen del caso
- Escalado automático al tamaño de la ventana

Capítulo 8

Stubs para algoritmos pendientes (algorithms/)

Cada stub compila y lanza `UnsupportedOperationException`. Los compañeros deben reemplazar el cuerpo del método con la implementación real.

Cuadro 8.1: Stubs de algoritmos pendientes.

Archivo	Firma del método	Complejidad esperada
Warshall.java	<code>static boolean[] [] transitiveClosure(...)</code>	$O(V^3)$
FloydWarshall.java	<code>static double[] [] shortestPaths(...)</code>	$O(V^3)$
Dijkstra.java	<code>static double[] shortestPath(...)</code>	$O((V + E) \log V)$
Kruskal.java	<code>static ArrayList<Edge> buildMST(...)</code>	$O(E \log E)$
DFS.java	<code>static ArrayList<Integer> preorder(...)</code>	$O(V + E)$

Capítulo 9

Modificaciones a archivos existentes

9.1. Renombrados (extensión .java)

Los siguientes archivos no tenían extensión .java y fueron renombrados:

- `ArrayList` → `ArrayList.java`
- `BFS` → `BFS.java`
- `Edge` → `Edge.java` (se eliminó el `Edge.java` vacío que existía)
- `HashMap` → `HashMap.java`
- `HashSet` → `HashSet.java`
- `LinkedList` → `LinkedList.java`
- `Map` → `Map.java`
- `MinHeap` → `MinHeap.java`
- `Prim` → `Prim.java`
- `Set` → `Set.java`

9.2. Package declarations

Se agregó `package cr.ac.tec.ce1103.logistec.graph`; a los 11 archivos existentes y se movieron a la subcarpeta `graph/`.

9.3. Import agregado a `LinkedList.java`

Se agregó `import java.util.Iterator;` para que la clase `LinkedListIterator` funcione correctamente (el `Iterator` es parte del JDK, no una estructura de datos del proyecto).

9.4. `Prim.java` — Nuevo constructor

Se agregó el constructor `Prim(HashMap<Integer, ArrayList>Edge<<existingGraph)` para permitir reutilizar un grafo ya cargado sin tener que volver a agregar aristas manualmente.

Capítulo 10

Caso de prueba (`test_cases/caso_base.json`)

10.1. Especificaciones

Cuadro 10.1: Especificaciones del caso de prueba.

Característica	Valor
Vértices	30 (distribuidos en cuadrícula 5×6)
Aristas	56 (calles horizontales, verticales y diagonales)
Paquetes	16 (prioridades 1–3, pesos 3–15 kg)
Camiones	3 (capacidades 40, 50 y 35 kg)
Depósito	BC (coordenadas 380, 170)
Grafo	Conexo (todos los vértices alcanzables)

10.2. Resultado esperado de la asignación best-fit

Cuadro 10.2: Resultado esperado de la asignación best-fit.

Camión	Capacidad	Carga asignada	Ocupación	Paradas
C01	40 kg	39 kg	97.5 %	AA, DE, BE, FC, CA
C02	50 kg	47 kg	94.0 %	EE, CE, AD, EB, FD
C03	35 kg	34 kg	97.1 %	CC, BD, AE, DA
Rechazados	—	13 kg	—	P09 (FA, 7 kg), P12 (FB, 6 kg)

Capítulo 11

Archivos de compilación y ejecución

11.1. compile.bat

```
1 javac -d out -cp "lib/gson.jar" -sourcepath src src\cr\ac\tec\ce1103\logistec\*.java ...
```

Listing 11.1: Script de compilación.

11.2. run.bat

```
1 java -cp "out;lib/gson.jar" cr.ac.tec.ce1103.logistec.LogisTEC test_cases\caso_base.json
```

Listing 11.2: Script de ejecución.

11.3. Estructura de directorios (raíz del proyecto)

```
Proyecto_Estructuras_final/
+-- src/cr/ac/tec/ce1103/logistec/    <- Codigo fuente
+-- out/                             <- Compilados (.class)
+-- lib/gson.jar                     <- Gson para parseo JSON
+-- test_cases/caso_base.json        <- Caso de prueba
+-- compile.bat                      <- Script de compilacion
+-- run.bat                          <- Script de ejecucion
+-- plan_fase3.md                    <- Plan de trabajo
+-- java-comment-style.md            <- Guia de estilo Javadoc
+-- fase3_documentation.md           <- Este documento
```

Capítulo 12

Dependencias externas

Cuadro 12.1: Dependencias externas del proyecto.

Librería	Versión	Propósito
Gson	2.11.0	Parseo de archivos JSON de configuración

Ubicación: `lib/gson.jar`

Capítulo 13

Estado de la compilación

```
1 $ javac -d out -cp "lib/gson.jar" -sourcepath src ... (22 archivos)
2 Compilacion exitosa -- 0 errores, 0 warnings
```

Listing 13.1: Resultado de la compilación.

13.1. Resultados de la ejecución de prueba

- Carga JSON: 30 vértices, 56 aristas, 16 paquetes, 3 camiones ✓
- Prim MST: 29 aristas, peso total 4450 ✓
- Best-fit: 14 paquetes asignados, 2 rechazados ✓
- Warshall: [stub] Pendiente de implementación
- FloydWarshall: [stub] Pendiente de implementación
- Kruskal: [stub] Pendiente de implementación
- NN + MST: rutas calculadas cuando FloydWarshall y DFS estén implementados

Capítulo 14

Responsabilidades del equipo

14.1. Pendiente (compañeros)

Cuadro 14.1: Algoritmos pendientes por implementar.

Algoritmo	Archivo	Depende de
Warshall	algorithms/Warshall.java	graph.HashMap, ArrayList, Edge
FloydWarshall	algorithms/FloydWarshall.java	graph.HashMap, ArrayList, Edge
Dijkstra	algorithms/Dijkstra.java	graph.HashMap, ArrayList, Edge

Cuadro 14.2: Algoritmos pendientes por implementar (continuación).

Algoritmo	Archivo	Depende de
Kruskal	algorithms/Kruskal.java	graph.UnionFind, ArrayList, Edge
DFS	algorithms/DFS.java	graph.HashMap, ArrayList

14.2. Ya implementado (en la Fase 3)

- Restructuración en paquetes
- Modelo de dominio (Package, Truck)
- Carga JSON (GraphLoader)
- Planificador (RoutePlanner con best-fit, NN, MST-based)
- Aplicación principal (LogisTEC)
- Interfaz gráfica Swing (LogisTECFrame)

- Caso de prueba (caso_base.json)